

CLUB club

2026-09-11

2026-09-11 class

Today we'll talk about and work on sequence data.

- FASTA files
- FASTQ files
 - What is the quality string?
 - Paired-end reads
- Sequencing read trimming and quality
- Editing text files
- Speed on the command line

FASTA

This is a very simple format, giving IDs and nucleotide (or amino acid) sequences:

```
>AB116084.1 Hepatitis B virus DNA (HBV-BAN81)
TTCCACAGCTTTCCACCAAGCCCTGCAAGATCCCAGAGTCAGGGG
CCTGTATTTTCCTGCTGGTGGCTCC
>X75657.1 Human hepatitis virus (genotype E, Bas)
TTCCACAACATTCCACCAAGCTCTG
CAGGATCCCAGAGTAAGAGG
GGTCAACTTTGACTGccAT
```

Sequences don't have to be the same length. Sequences may span multiple lines. IDs might be duplicated. There might be ambiguous codes (e.g. **W** can mean **A** or **T**. See https://en.wikipedia.org/wiki/Nucleic_acid_notation)

FASTQ

A more complicated format, giving IDs, nucleotide sequences, and a "PHRED" quality score per nucleotide. FASTQ are produced from sequencer output, in which the sequencer infers each nucleotide. It provides well-calibrated estimates of nucleotide "call" accuracy.

Here's an example FASTQ record for a single read:

```
@HWI-D00230L:378:CAGUJANXX:2:2116:14396:83860
TCTTTCGGAGTGTGGATTCGCACTCCTCCAGCATATAGAT
+
CBCBBGGG/CF1EE<;FFG@<CEGGGGBGGGGGG>FGFD?
```

As with FASTA, sequences may differ in length, may span multiple lines, and IDs might be duplicated. The ID can optionally be repeated after the `+` (see Wikipedia).

Ambiguous codes are rarely used. The sequencer makes its best guess and gives you a quality score.

FASTQ has a low-frequency bad gotcha: the first character on a line of quality scores can be a `@`, which looks like the start of the next read!

PHRED quality scores

```
@HWI-D00230L:378:CAGUJANXX:2:2116:14396:83860
TCTTTCGGAGTGTGGATTGCGACTCCTCCAGCATATAGAT
+
CBCBBGGG/CF1EE<;FFG@<CEGGGBGGGGGG>FGFD?
```

The quality score for each nucleotide "call" is -10 times the (base 10) logarithm of the probability the sequencer estimates it has made a mistake (converted to a natural number). Sounds complicated, but is actually simple. See <https://www.desmos.com/calculator/r4lawpiq4n>

■ Example: the ? in the above FASTQ record

Suppose the sequencer estimates the probability it has made a wrong nucleotide "call" as $1/1000$. This is 10^{-3} . The quality score will be this times -10 . I.e., 30 .

The 30 is then added to 33. The 33 corresponds to `!`, which is the first printing character in the ASCII code. This gives 63, which is the ASCII value for `?` (see [man ascii](#)).

This quality encoding scheme is sometimes also called **PHRED+33**.

■ In summary

The nucleotide quality scores for a read sequence are given as a compact character string encoding the estimated per-nucleotide error probabilities.

See https://en.wikipedia.org/wiki/Phred_quality_score

ASCII

From `man ascii`

0	nul	1	soh	2	stx	3	etx	4	eot	5	enq	6	ack	7	bel
8	bs	9	ht	10	nL	11	vt	12	np	13	cr	14	so	15	si
16	dle	17	dc1	18	dc2	19	dc3	20	dc4	21	nak	22	syn	23	etb
24	can	25	em	26	sub	27	esc	28	fs	29	gs	30	rs	31	us
32	sp	33	!	34	"	35	#	36	\$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	\	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124		125	}	126	~	127	del

Sources of FASTQ data

- Retrieve using `lftp` from a Charite "filebox" URL. `pixi add lftp`
- SRA download
- Sequencing download
- Downloaded via your browser
- Someone passes it to you on a USB stick
- Etc.

How to transfer files from fileboxes

File Boxes (<https://file-exchange.bihealth.org/>) provide encrypted transfer of big files. Multiple routes of access are available (<https://bihealth.github.io/bih-cluster/how-to/service/file-exchange>). You can log in with your Charite (`user@CHARITE`) or MDC account (`user@MDC-BERLIN`) to access it.

Access via command line

Download a file via one file url using wget

```
wget --user=user@MDC-BERLIN --ask-password url
```

Example:

```
wget --user=johnc@MDC-BERLIN --ask-password https://file-exchange.bihealth.org/c31121dd-bf66-4248-ac67-c9ab8f18ca84/P12341S1L001R1_001.fastq.gz
```

Download many *.gz files using a filebox url using wget

```
wget -rnd -np --user=user@MDC-BERLIN --ask-password -A '*.gz' url
```

Example:

```
wget -rnd -np --user=johnc@MDC-BERLIN --ask-password -A '*.gz' https://file-exchange.bihealth.org/de971ejlkjsi96-4aaf-ae85-c92e6fff38e6/
```

Using lftp

```
cd ../targetfolder\  
lftp\  
open -u user@MDC-BERLIN url\  
mget -c * &
```

Example:

```
open -u johnc@MDC-BERLIN https://file-exchange.bihealth.org/de971ejlkjsi96-4aaf-ae85-c92e6fff38e6/
```

NGS pipeline

■ Preliminary steps

- Make an index from some reference sequences
- Get the sequencing data

■ Processing the NGS data

- Demultiplex (if necessary) to obtain FASTQ "reads"
- Look at the quality report - did the sequencing work ok?
- Trim the reads
- Align the reads to the references
- Examine the results

Today we will trim and look at quality.

Also: keep in mind my comment from last week that there are often many programs for each possible step.

Trimming reads

We will use `fastp` (see <https://github.com/OpenGene/fastp>).

Install it:

```
$ pixi add fastp
✓ Added fastp >=1.3.6,<2
```

Then make yourself a new shell and confirm that `fastp` is installed and found:

```
pixi shell
fastp --help
```

The `--help` option produces a lot of output. You can see it a page at a time via:

```
fastp --help | less
```

Some FASTQ data to work on

I've made a `data` directory in our shared storage area, and in that a sub-directory called `fastq`:

```
$ cd /sc-projects/sc-proj-cc11-civclub/data
$ ls -lh
total 11624
-rwxrwxr--+ 1 jonestc posix-nogroup 425K Sep 10 20:58 sample_1_R1_001.fastq.gz
-rwxrwxr--+ 1 jonestc posix-nogroup 417K Sep 10 20:58 sample_1_R2_001.fastq.gz
-rwxrwxr--+ 1 jonestc posix-nogroup 4.3M Sep 10 20:58 sample_2_R1_001.fastq.gz
-rwxrwxr--+ 1 jonestc posix-nogroup 4.2M Sep 10 20:58 sample_2_R2_001.fastq.gz
```

What is the .gz suffix on those file names?

The `.gz` indicates that the file is compressed with `gzip`.

- `gzip` can be used to compress a file (this adds the `.gz` suffix)
- `gunzip` will uncompress a file

There are many compression programs. E.g., `zip` (installed by default, uses a `.zip` suffix) and `bzip` (install with `pip install bzip2`; uses a `.bz2` suffix)).

Importantly: `gzip` and `gunzip` read from "standard input" and write to "standard output" if you don't specify files for them to work on.

```
# Print the first 8 lines of sample_1_R1_001.fastq.gz
$ gunzip < sample_1_R1_001.fastq.gz | head -n 8
```

Notice that the file is still compressed on disk.

```
# How many bytes (characters) is sample_2_R1_001.fastq.gz
# when uncompressed?
$ gunzip < sample_2_R1_001.fastq.gz | wc -c
27511492
```

```
# How many reads does sample_2_R1_001.fastq.gz contain?
# Caution: remember what I said about FASTQ having a gotcha!
$ gunzip < sample_2_R1_001.fastq.gz | egrep -c '^@'
100000
```

Running fastp

The following works for me because I have a symbolic link in my home directory () called `data` that points to `/sc-projects/sc-proj-cc11-civclub/data`:

```
$ mkdir 20260911-ngs-intro
$ cd 20260911-ngs-intro

$ time fastp \
  -i ~/data/fastq/sample_1_R1_001.fastq.gz \
  -I ~/data/fastq/sample_1_R2_001.fastq.gz \
  -o sample_1_R1-trimmed.fastq.gz \
  -O sample_1_R2-trimmed.fastq.gz \
  -j sample_1.json \
  -h sample_1.html
```

You can put `time` in front of any command, and the time taken to run the command will be printed after it finishes.

Shell scripts

You can put commands into a file. It's easier to enter/edit them there, and of course it's easier to re-execute them.

```
$ cat trim_1.sh
time fastp \
  -i sample_1_R1_001.fastq.gz \
  -I sample_1_R2_001.fastq.gz \
  -o sample_1_R1-trimmed.fastq.gz \
  -O sample_1_R2-trimmed.fastq.gz \
  -j sample_1.json \
  -h sample_1.html
```

The `trim_1.sh` and `trim_2.sh` scripts are also in ...

Text editing

`nano` is working quite well for me now. I changed to a dark theme in Jupyter because the default highlighting looked bad with the light theme.

But... you can also edit files using the Jupyter interface (i.e., not from the shell using `nano`).

If you're getting the impression that I don't know much about using Jupyter Hub, you're right! Please explore, learn, and help us all to improve.

Command-line speed

As I mentioned, you really need to learn some command line shortcuts to massively increase the speed at which you can do things.

You may need to change the Terminal settings to make the `Option` key work like `Alt` (for those on a Mac, and maybe others?), as used in the following list:

■ The command line is a one-line text editor

- `Control-a` takes you to the beginning of the line
- `Control-e` takes you to the end of the line
- `Control-r` incremental reverse search of your command history
- `Control-y` yank back killed (deleted text)
- `Alt-.` insert the last word from the previous command
- `Control-k` kill (delete) the characters to the end of the line
- `Control-f` move forward one character
- `Alt-f` move forward one word
- `Control-b` move backward one character
- `Alt-b` move backward one word
- `Control-d` delete one character
- `Alt-d` delete one word (after the cursor)
- `Alt-backspace` delete the word before the cursor